Progress Report

Comparing LLM-Based Policy Generation and Reinforcement Learning for Embodied Control

Jarne Demoen

Week 1: Initial Pipeline and Baseline Evaluation

0.1 Goal of the Week

The goal of Week 1 was to create a minimal working experimental pipeline for the project. Instead of immediately starting with LLM-generated controllers, I first focused on a simple continuous-control environment. The main objective was to verify that I could create an environment, evaluate fixed controllers, train a reinforcement learning baseline, and save the results in a structured way.

0.2 Environment

The first environment used was `Pendulum-v1` from Gymnasium. I selected this environment because it has continuous observations and continuous actions. This makes it more relevant to the embodied-control setting of the project than a discrete task such as `CartPole`, while still being simple enough for debugging.

The observation contains three values:

- $\cos(\theta)$, the cosine of the pendulum angle;
- $\sin(\theta)$, the sine of the pendulum angle;
- $\dot{\theta}$, the angular velocity.

The action is one continuous torque value in the interval $[-2, 2]$. The goal is to apply torque so that the pendulum reaches and stays near the upright position. In this environment, rewards are usually negative, so values closer to zero indicate better performance.

0.3 Implemented Methods

Three methods were considered in the Week 1 report: a random controller, a manually defined controller, and a PPO baseline.

0.3.1 Random Controller

The random controller samples a torque uniformly from the valid action range at each time step. It does not use the observation and does not learn. It is included as a lower baseline to check whether other methods can do better than random actions.

0.3.2 Manual Controller

The manual controller is a fixed hand-written rule. It uses the pendulum angle and angular velocity to compute a torque. The angle is reconstructed from the observation using

$$\theta = \text{atan2}(\sin(\theta), \cos(\theta)).$$

The controller then applies a simple proportional-derivative style rule:

$$u = -2.0\theta - 0.5\dot{\theta}.$$

Here, u is the torque applied to the pendulum. The value of u is clipped to the valid torque range $[-2, 2]$. This controller is not trained; it is only a simple manually designed policy.

0.3.3 PPO Baseline

Proximal Policy Optimization, or PPO, is a policy-gradient reinforcement learning algorithm introduced by Schulman et al. [5]. It learns a parameterized policy $\pi_\theta(a \mid s)$, where s is the current state, a is the selected action, and θ represents the parameters of the neural network policy. During training, the agent interacts with the environment, collects trajectories, estimates which actions were better or worse than expected, and updates the policy.

The main idea behind PPO is to improve the policy without making updates that are too large. Large policy updates can make training unstable. PPO therefore compares the probability of an action under the new policy with the probability of the same action under the old policy. This probability ratio is defined as

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

PPO optimizes a clipped objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t \right) \right].$$

Here, $\hat{A}_t$ is the estimated advantage and ϵ is the clipping parameter. The advantage indicates whether an action was better or worse than expected. The clipping term limits how much the new policy can deviate from the old policy during one update. This makes PPO relatively stable and easy to use compared to older policy-gradient methods. The first reinforcement learning baseline was trained using Proximal Policy Optimization, implemented with Stable-Baselines3. PPO was included to check whether the pipeline can compare fixed controllers with a learned policy.

0.4 Evaluation Setup

The random and manual controllers were evaluated on `Pendulum-v1` for 100 episodes using seed 42. The PPO model was trained for 500,000 timesteps and also evaluated for 100 episodes. The reported metric is the mean cumulative reward over the evaluation episodes. Since rewards in `Pendulum-v1` are negative, higher values are better.

0.5 Results

Table 1 shows the Week 1 results.

Table 1: Week 1 results on `Pendulum-v1`. Higher reward is better.

Method	Episodes	Mean reward	Std. reward	Training
Random controller	100	−1239.407	293.894	No
Manual controller	100	−1180.920	234.175	No
PPO baseline	100	−182.502	107.747	Yes

The manual controller performed better than the random controller. The improvement in mean reward shows that the manually defined rule uses some useful information from the state. However, the improvement is limited. The PPO baseline obtained a mean reward of −182.502, which is substantially better than both fixed controllers. The current ranking is therefore

$$\text{PPO} > \text{Manual controller} > \text{Random controller}.$$

0.6 Comparison with Reported PPO Results

To put the obtained PPO result into context, I compared it with a publicly available Stable-Baselines3 PPO model for `Pendulum-v1` hosted on Hugging Face [6]. The reported result for that model is a mean reward of −230.423 with a standard deviation of 142.539, evaluated deterministically over 10 episodes. In my pipeline, the PPO baseline obtained a mean reward of −182.502 with a standard deviation of 107.747. This is better than the reported Stable-Baselines3 result, since rewards in `Pendulum-v1` are negative and values closer to zero indicate better performance.

0.7 Discussion

The Week 1 results show that the experimental pipeline is functioning correctly. The random controller provides a useful lower baseline because it does not use the state and simply samples arbitrary actions. Its poor mean reward confirms that `Pendulum-v1` cannot be solved well by uncontrolled torque selection. The manual controller performs slightly better, indicating that even a simple rule based on angle and angular velocity can leverage meaningful information from the environment. However, the improvement over random control is relatively small. This suggests that manually designing a good controller is not trivial, even in a simple environment.

The PPO baseline performs substantially better than both fixed controllers. This is expected because PPO can improve its policy through interaction with the environment, while the random and manual controllers remain fixed. The large gap between PPO and the manual controller also illustrates why reinforcement learning is a strong baseline for embodied-control tasks. Instead of relying on a manually chosen formula, PPO can adapt the policy parameters based on reward

feedback. The comparison with the reported Stable-Baselines3 PPO result further supports this conclusion, where my PPO baseline performs better in this specific comparison. Nevertheless, the evaluation settings are not identical: the reported result uses 10 evaluation episodes, while my result uses 100 episodes. Because of this, the comparison should be treated as evidence that the implementation is reasonable and competitive, not as a definitive claim that my trained model is superior. At the same time, the results show why comparing with LLM-based policy generation will be interesting. A manually written controller is easy to interpret, but its performance is limited. PPO gives much better performance, but the learned policy is less transparent because it is represented by neural network parameters. LLM-generated controllers may offer a different trade-off: they could produce readable control logic while still using more task knowledge than a very simple hand-written rule. Therefore, the next experiments should not only compare rewards but also consider factors such as interpretability, ease of modification, and the amount of human intervention needed.

Overall, Week 1 successfully establishes the basic structure needed for the project. The environment can be created, controllers evaluated, PPO trained, and results stored and compared. This creates a foundation for the next phase, where LLM-based policy generation can be added to the same evaluation pipeline.

0.8 Conclusion

In Week 1, I implemented and evaluated a first control pipeline for `Pendulum-v1`. The pipeline includes a random controller, a simple manually defined controller, and a PPO reinforcement learning baseline. The results show a clear performance ranking: PPO performs best, followed by the manual controller, while the random controller performs worst. The main technical outcome is that the pipeline is working end-to-end. I can now evaluate fixed controllers, train a reinforcement learning agent, compute mean and standard deviation over multiple episodes, and compare the results in a structured way. This is an important first step because future LLM-generated controllers can be evaluated using the same setup. The results also clarify the role of the different baselines. The random controller serves as a lower bound, the manual controller represents a simple interpretable policy, and PPO represents a strong learned reinforcement learning baseline.

Week 2: Reinforcement Learning Baselines and Reproducibility

0.9 Goal of the Week

The goal of Week 2 was to extend the initial reinforcement learning pipeline by implementing or configuring three reinforcement learning baselines: PPO, SAC, and DDPG. In Week 1, PPO was already implemented and evaluated on `Pendulum-v1`. In Week 2, the focus was therefore on adding SAC and DDPG, running initial experiments on the same simple continuous-control environment, defining reward thresholds, and verifying that experiments can be reproduced using fixed seeds.

0.10 Implemented Reinforcement Learning Baselines

Three reinforcement learning algorithms are now part of the baseline pipeline: PPO, SAC, and DDPG. These methods were selected because they are commonly used for continuous-control problems and can be applied directly to environments with continuous action spaces such as

`Pendulum-v1`. Although all three methods learn policies from reward feedback, they differ strongly in how they collect data, how they update the policy, and how they handle exploration.

0.10.1 Soft Actor-Critic

Soft Actor-Critic, or SAC, was added as a second baseline. SAC is an off-policy actor-critic algorithm based on the maximum entropy reinforcement learning framework [2]. Unlike PPO, SAC stores previous transitions in a replay buffer and can reuse them for multiple gradient updates. This makes SAC more sample-efficient, especially in continuous-control environments. SAC learns a stochastic policy, a value-based critic, and usually uses target networks for stable learning. The actor outputs a probability distribution over continuous actions. The critic estimates the expected quality of state-action pairs using a Q-function:

$$Q(s, a).$$

The policy is trained not only to maximize expected return, but also to maximize entropy. Entropy measures how random or uncertain the policy is. The SAC objective can be understood as maximizing both reward and policy entropy:

$$\mathbb{E}\left[\sum_t r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot \mid s_t))\right],$$

where $\mathcal{H}$ is the entropy of the policy and α controls the importance of the entropy term. This means that SAC is rewarded for achieving high task reward while still keeping the policy sufficiently exploratory. This entropy term is one of the main differences between SAC and DDPG. In DDPG, exploration is usually added externally through noise on the actions. In SAC, exploration is part of the objective itself. This often makes SAC more robust, because the policy is explicitly encouraged to avoid becoming deterministic too early.

For `Pendulum-v1`, this is useful because the agent must discover how to swing the pendulum upward and then stabilize it near the top. A policy that becomes too deterministic too early may converge to poor behavior. SAC's entropy term helps the agent continue exploring different torque values during training, which can improve learning in continuous action spaces.

0.10.2 Deep Deterministic Policy Gradient

Deep Deterministic Policy Gradient, or DDPG, was added as a third baseline. DDPG is also an off-policy actor-critic algorithm for continuous action spaces [3, 4]. The actor in DDPG directly maps a state to one action:

$$\mu_\theta(s) = a.$$

For `Pendulum-v1`, this means that the policy directly outputs a torque value for the current pendulum state. The critic learns a Q-function $Q(s, a)$ that estimates how good a given action is in a given state. The actor is then updated in the direction that increases the critic's estimated Q-value:

$$\nabla_\theta J(\theta) \approx \mathbb{E}\left[\nabla_a Q(s, a)|_{a=\mu_\theta(s)} \nabla_\theta \mu_\theta(s)\right].$$

This is called the deterministic policy gradient. Instead of learning a probability distribution over actions, DDPG learns which specific continuous action should be selected.

Because the policy is deterministic, DDPG needs an external mechanism for exploration during training. This is usually done by adding noise to the selected action. The policy may output one torque value, but noise is added so the agent tries slightly different actions. This helps the agent discover better behavior, but it also makes DDPG sensitive to the choice of exploration noise and other hyperparameters. DDPG also uses target networks for the actor and critic. These target networks are slowly updated versions of the main networks and are used to make the Q-learning targets more stable. Without target networks, the critic target could change too quickly, which can destabilize training.

DDPG is relevant for this project because it is specifically designed for continuous action spaces. However, compared with SAC, it is generally considered more sensitive. SAC improves on some weaknesses of DDPG by using a stochastic policy and entropy regularization, which often leads to more stable exploration.

0.10.3 Similarities and Differences

PPO, SAC, and DDPG all use neural networks to learn control policies from interaction with the environment. They all receive observations from `Pendulum-v1`, output continuous torque actions, and use reward feedback to improve performance. They are also all actor-critic methods, meaning that they combine a policy component with a value-estimation component. The main difference is how they use data. PPO is on-policy, so it learns from data collected by the current policy. SAC and DDPG are off-policy, so they can store old transitions in a replay buffer and reuse them. This usually makes SAC and DDPG more sample-efficient than PPO.

A second important difference is the type of policy. PPO and SAC use stochastic policies during training. They learn a distribution over actions. DDPG uses a deterministic policy, directly mapping each state to a specific action. Because of this, DDPG needs external exploration noise, while SAC includes exploration directly in the learning objective through entropy maximization.

A third difference is stability. PPO is often stable because its clipped objective prevents the policy from changing too much in one update. SAC is often strong and stable in continuous-control tasks because it combines off-policy learning with entropy-based exploration. DDPG can perform well, but it is usually more sensitive to hyperparameters, critic errors, and exploration noise.

0.11 Experimental Setup

All Week 2 experiments were run on `Pendulum-v1`. This keeps the comparison consistent with Week 1 and makes it easier to verify whether the different reinforcement learning baselines behave as expected on a simple continuous-control task.

The PPO result is taken from the Week 1 experiment, where PPO was trained for 500,000 timesteps and evaluated for 100 episodes. SAC and DDPG were trained for 20,000 timesteps as initial experiments. The same evaluation principle was used: the reported metric is the mean cumulative reward, together with the standard deviation. Since rewards in `Pendulum-v1` are negative, values closer to zero indicate better performance.

To support reproducibility, fixed seeds were used in the experimental pipeline. This is important because reinforcement learning results can vary due to random initialization, environment

randomness, exploration behavior, and stochastic training updates. Using fixed seeds makes it easier to debug the pipeline and to check whether repeated runs produce comparable results.

0.12 Reward Thresholds

To interpret the results more clearly, I defined simple reward thresholds for `Pendulum-v1`. These thresholds are not official success criteria, but they provide a practical way to classify the quality of the learned policies during the project.

- Mean reward below -1000: poor performance. The controller is close to random or does not solve the task meaningfully.
- Mean reward between -1000 and -500: limited performance. The controller shows some improvement but is still far from a good policy.
- Mean reward between -500 and -250: reasonable performance. The controller has learned useful behavior, but there is still room for improvement.
- Mean reward above -250: strong performance. The controller performs well on the task.

Using these thresholds, the random and manual controllers from Week 1 are classified as poor, while PPO, SAC, and DDPG all reach the strong-performance range.

0.13 Results

Table 2 shows the current reinforcement learning baseline results on `Pendulum-v1`.

Table 2: Week 2 reinforcement learning baseline results on `Pendulum-v1`. Higher reward is better.

Method	Total timesteps	Mean reward	Std. reward
PPO	500,000	-182.502	107.747
SAC	20,000	-149.033	73.988
DDPG	20,000	-153.602	76.448

The results show that all three reinforcement learning baselines perform much better than the random and manual controllers from Week 1. SAC obtained the best mean reward, with a score of -149.033. DDPG followed closely with a mean reward of -153.602. PPO obtained a mean reward of -182.502.

The current ranking among the reinforcement learning baselines is therefore

$$\text{SAC} > \text{DDPG} > \text{PPO}.$$

However, this ranking should be interpreted carefully because the training budgets are not identical. PPO was trained for 500,000 timesteps, while SAC and DDPG were trained for 20,000 timesteps. Therefore, the table is useful as an initial comparison of working baselines, but it is not yet a fully controlled benchmark. A more rigorous comparison should train all algorithms under the same budget and preferably repeat each experiment over multiple seeds.

0.14 Discussion

The Week 2 results show that the reinforcement learning baseline pipeline has been successfully extended beyond PPO. Both SAC and DDPG were configured and trained on the same environment, and both obtained strong results according to the defined reward thresholds. This is an important step because the project now has multiple learned baselines instead of relying on a single reinforcement learning method.

SAC achieved the best result in the current experiments. This is plausible because SAC is often strong in continuous-control tasks due to its off-policy learning and entropy-based exploration. Since SAC stores transitions in a replay buffer, it can reuse previous environment interactions for several training updates. This makes it more sample-efficient than PPO, which relies on fresh on-policy trajectories. The entropy term also encourages the policy to keep exploring different torque values instead of becoming too deterministic too early. In a task such as `Pendulum-v1`, where the agent must learn both swing-up and stabilization behavior, this can be beneficial.

DDPG also performed well, with a mean reward close to SAC. This shows that a deterministic actor-critic approach can also learn useful continuous-control behavior for `Pendulum-v1`. DDPG is conceptually different from SAC because its actor directly outputs one action instead of a probability distribution. This can be efficient once a good policy has been found, but it also means that exploration must be added separately through action noise. Because of this, DDPG can be more sensitive to the exploration strategy and to hyperparameter choices.

PPO remains a useful baseline, even though it did not obtain the best mean reward in this comparison. PPO is stable and simple to use because its clipped objective limits the size of policy updates. This reduces the risk of destructive policy changes during training. However, PPO is on-policy, meaning that it cannot reuse old experience in the same way as SAC and DDPG. This can make PPO less sample-efficient, especially when environment interaction is expensive.

The comparison between PPO, SAC, and DDPG is therefore not only a comparison of reward values. It also compares three different reinforcement learning design choices. PPO uses stable on-policy updates, SAC uses off-policy learning with stochastic maximum-entropy exploration, and DDPG uses off-policy learning with a deterministic policy and external exploration noise. These differences matter for the project because LLM-generated controllers will later be compared against reinforcement learning baselines. A strong comparison should therefore include more than one RL algorithm.

The main limitation of the Week 2 comparison is that the training budgets are different. PPO was trained for 500,000 timesteps, while SAC and DDPG were trained for 20,000 timesteps. Therefore, the current results should not be interpreted as a definitive algorithm ranking. Instead, Week 2 should be seen as an implementation and verification phase. The key outcome is that PPO, SAC, and DDPG all run correctly, produce meaningful results, and can be evaluated using the same pipeline. A more rigorous comparison should train all algorithms with the same interaction budget and repeat each experiment over multiple seeds. This would make it possible to compare not only the average reward, but also the stability and variance of each method. This is especially important for DDPG, because deterministic actor-critic methods can be sensitive to random initialization and exploration noise.

0.15 Conclusion

In Week 2, I extended the experimental pipeline by adding SAC and DDPG alongside the PPO baseline from Week 1. All three reinforcement learning algorithms were evaluated on

`Pendulum-v1`. The results show that SAC, DDPG, and PPO all achieve strong performance according to the defined reward thresholds. The best result in the current experiments was obtained by SAC, followed closely by DDPG. PPO also remained clearly stronger than the fixed random and manual controllers from Week 1. These results confirm that the reinforcement learning part of the project is functioning correctly and that the pipeline can support multiple baseline algorithms.

The main next step is to make the comparison more systematic. Future experiments should use equal training budgets for PPO, SAC, and DDPG, evaluate each method over multiple seeds, and then compare the algorithms more fairly. After that, LLM-generated controllers can be added to the same evaluation setup and compared against these reinforcement learning baselines.

Week 3: One-Shot LLM Controller Generation

0.16 Goal of the Week

The goal of Week 3 was to implement the first LLM-based controller generation pipeline. In contrast to the reinforcement learning baselines from Week 1 and Week 2, the objective was not to train a neural policy through gradient-based optimization. Instead, the goal was to prompt a large language model to generate an executable controller directly from a textual description of the environment, the observation vector, the action vector, and the task objective.

This week focused specifically on one-shot controller generation. This means that the LLM generated a policy once, without receiving reward feedback or sensory-motor traces for iterative refinement. The expected output was a working pipeline that can generate controller code, execute it safely, evaluate it in `Pendulum-v1`, and store the generated policies, prompts, responses, evaluation logs, and reward results.

0.17 Relation to the Carvalho and Nolfi Method

The implementation follows the first phase of the method proposed by Carvalho and Nolfi [1]. In their approach, an LLM receives a textual description of the agent, its sensors, actuators, environment, and task objective. The model then generates a control policy that maps continuous observation vectors to continuous action vectors. For this project, `Pendulum-v1` is a suitable first environment because it has exactly this structure. The observation vector is

$$o = [\cos(\theta), \sin(\theta), \dot{\theta}],$$

where $\cos(\theta)$ and $\sin(\theta)$ encode the direction of the pendulum and $\dot{\theta}$ represents angular velocity. The action is a one-dimensional continuous torque value in the range $[-2, 2]$. The generated controller must therefore implement a function of the form

$$\pi(o) = a,$$

where o is the continuous observation vector and a is the continuous action vector. In code, this corresponds to a Python function with the following structure:

```
def policy(obs):
...
return [torque]
```

Following Carvalho and Nolfi's structured prompting strategy, the LLM was not asked to directly produce Python code. Instead, the prompting pipeline was divided into three stages:

1. Generate a high-level control strategy.
2. Translate the strategy into IF-THEN-ELSE control rules.
3. Convert the rules into executable Python code.

This step-by-step structure was used to encourage the model to reason about the task before producing executable controller code.

0.18 Implementation

The one-shot pipeline was implemented as a local controller generation and evaluation loop. The LLM was accessed through a local Ollama server using the model `llama3.1:8b`. This choice was motivated by the available hardware. Unlike Carvalho and Nolfi, who used large quantized models on an NVIDIA DGX cluster, this project was run on a single Ubuntu workstation with an RTX 4060 GPU. Therefore, the goal of Week 3 was not to reproduce their computational scale, but to implement the same general policy-generation workflow under local resource constraints.

For each run, the pipeline performs the following steps:

1. Create the Pendulum task prompt.
2. Ask the LLM for a high-level control strategy.
3. Ask the LLM to convert the strategy into IF-THEN-ELSE rules.
4. Ask the LLM to convert the rules into Python code.
5. Extract the generated Python code from the LLM response.
6. Save the generated policy to disk.
7. Run safety checks on the generated code.
8. Evaluate the policy in `Pendulum-v1`.
9. Store the evaluation result, generated code, prompts, responses, and sensory-motor traces.

Because the generated Python code is produced by an LLM, safety checks were required before execution. The implemented checks include rejecting unsafe imports, preventing the use of dangerous functions such as `eval`, `exec`, `open`, and `input`, executing policies with a timeout, verifying the returned action shape, and clipping the output torque to the valid range $[-2, 2]$.

0.19 Generated Policy Example

One generated policy used the following structure:

```
import math

def policy(obs):
cos_theta = float(obs[0])
sin_theta = float(obs[1])
theta_dot = float(obs[2])

theta = math.atan2(sin_theta, cos_theta)

if abs(theta) > math.pi/3:
    torque = -1 * math.sin(theta)
elif abs(theta) < math.pi/6:
    torque = 0.5 * math.sin(theta)
else:
    torque = -2.0 * theta - 0.5 * theta_dot

torque = max(-2.0, min(2.0, torque))

return [float(torque)]
```

This policy correctly reconstructs the angle using

$$\theta = \text{atan2}(\sin(\theta), \cos(\theta)),$$

produces a continuous torque value, and clips the torque to the valid action range. This shows that the LLM was able to generate syntactically valid and executable controller code. However, the quality of the generated control logic was limited, as shown by the reward results.

0.20 Experimental Setup

The one-shot LLM controllers were evaluated on `Pendulum-v1`. Each generated policy was evaluated for 10 episodes. Since each episode has a maximum length of 200 steps, each run used approximately 2,000 environment steps for evaluation.

To study the effect of sampling stochasticity, three temperature values were tested:

$$T \in 0.0, 0.4, 0.8.$$

For each temperature, five one-shot policies were generated. This resulted in 15 generated policies in total. A run was considered valid if the generated policy executed without runtime failures. If runtime failures occurred, the run was marked separately and excluded from the valid-run performance summary.

0.21 Results

Table 3 summarizes the one-shot LLM results. The reported mean reward is computed across the valid generated policies for each temperature. Higher reward is better.

Table 3: Week 3 one-shot LLM controller results on `Pendulum-v1`. Higher reward is better.

Temperature	Valid runs	Mean reward	Std. across runs	Best run
0.0	5/5	−1202.858	70.189	−1105.627
0.4	4/5	−1287.799	36.489	−1247.412
0.8	4/5	−1522.251	233.701	−1298.382

The best one-shot policy was obtained at temperature 0.0, with a mean reward of −1105.627 and a standard deviation of 361.933 over 10 evaluation episodes. All five runs at temperature 0.0 were valid. At temperature 0.4, four out of five runs were valid, and one run produced runtime failures. At temperature 0.8, four out of five runs were valid, and one run produced a large number of runtime failures.

0.22 Discussion

The Week 3 results show that the one-shot LLM controller pipeline is working end-to-end. The system can generate prompts, query a local LLM, extract Python code, execute the generated policy safely, evaluate it in `Pendulum-v1`, and store the results. This satisfies the main technical objective of the week. However, the one-shot policies are not competitive with the reinforcement learning baselines. The best one-shot LLM policy obtained a mean reward of −1105.627, while the PPO, SAC, and DDPG baselines all achieved substantially better performance in previous experiments. According to the reward thresholds defined in Week 2, the one-shot LLM policies are still in the poor-performance range. The results also show that temperature has an important effect on reliability. Temperature 0.0 produced the best and most stable results, with all generated policies executing successfully. Higher temperatures produced more variation, but this did not improve performance. Instead, the generated policies became less reliable and, in some cases, produced runtime failures. This suggests that, for a small local model such as `llama3.1:8b`, deterministic or low-temperature generation is more suitable for controller code generation.

Inspection of the generated policies helps explain the weak performance. The LLM often produced plausible-looking rules, but the resulting controllers did not consistently use angular velocity in all relevant situations. For example, some policies used angular velocity only in one branch of the rule system while ignoring it when the pendulum was far from upright or close to upright. This limits the controller's ability to perform swing-up and stabilization effectively. Pendulum control requires not only correcting the angle, but also exploiting and damping angular velocity. These results are still useful because they establish a one-shot LLM baseline. The poor performance of one-shot generation motivates the next phase of the project: iterative policy refinement. In the Carvalho and Nolfi method, the LLM does not only generate an initial controller; it also receives performance feedback and sensory-motor data from previous evaluations and is prompted to improve the policy. The Week 3 pipeline already stores sensory-motor traces, which can be used in Week 4 to implement this refinement loop.

0.23 Conclusion

In Week 3, I implemented the first LLM-based controller generation pipeline. The system generates a high-level strategy, converts it into control rules, produces executable Python code, applies safety checks, evaluates the generated policy in `Pendulum-v1`, and stores the resulting logs. The main outcome is that the one-shot LLM pipeline works technically. Generated controllers can be executed and evaluated without manual rewriting. The best valid one-shot policy achieved a mean reward of -1105.627, which is far weaker than the reinforcement learning baselines. Therefore, one-shot LLM policy generation with the local `llama3.1:8b` model is not sufficient to solve the task well.

The next step is to implement iterative policy refinement. In this phase, the LLM will receive the current policy, the best policy so far, reward scores, and sensory-motor data from evaluation. This should test whether feedback-based prompting can improve the generated policies beyond the initial one-shot results.

References

[1] Carvalho, A., and Nolfi, S. Comparing LLM-Generated Policies and Reinforcement Learning for Embodied Control. In Proceedings of the International Conference on Learning Representations, 2026.

[2] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. arXiv preprint arXiv:1801.01290, 2018.

[3] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. Continuous Control with Deep Reinforcement Learning. arXiv preprint arXiv:1509.02971, 2015.

[4] OpenAI Spinning Up. Deep Deterministic Policy Gradient. Available at: https://spinningup.openai.com/en/latest/algorithms/ddpg.html.

[5] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347, 2017.

[6] Stable-Baselines3. PPO Agent Playing `Pendulum-v1`: Results JSON. Hugging Face, 2024. Available at: https://huggingface.co/sb3/ppo-Pendulum-v1/blob/main/results.json.

[7] Demoen, J. Comparing LLM-Based Policy Generation and Reinforcement Learning for Embodied Control. Estudo Dirigido Project Proposal, 2026.